

实验十四 综合实验

（一）目的和要求

- （1）掌握深度学习的通用流程
- （2）进一步理解卷积神经网络结构和核心网络层的工作原理
- （3）理解迁移学习及生成对抗神经网络的工作原理
- （4）能够使用 Keras 的 ImageDataGenerator 模块进行数据增强，并构建卷积神经网络模型
- （5）能够保存训练好的网络模型，并能将保存的网络模型实例化
- （6）会配置训练网络模型过程中的各种参数

（二）实验内容及步骤

本次实验提供了迁移学习、生成对抗神经网络及卷积神经网络的三个具体应用。从下述三个小实验中，自选两个实验完成本次实验任务。

（1）猫狗大战

1. 下载并整理图像

步骤 1 访问 <https://www.kaggle.com/c/dogs-vs-cats>，在打开的网页中单击“Data”→“Download All”按钮，下载图像压缩包。

步骤 2 解压下载的文件“dogs-vs-cats.zip”，将“train.zip”和“test1.zip”解压到 Jupyter Notebook 的工作目录下。

步骤 3 为方便建立图像标签，在文件夹“train”中建立两个子文件夹，分别命名为“cat”和“dog”，并将文件夹“train”中文件名以“cat”开头的图像文件移动到文件夹“cat”中，将文件名以“dog”开头的图像文件移动到文件夹“dog”中。

2. 构建数据集

步骤 1 导入本项目所用模块与包。

步骤 2 定义数据读取函数，函数参数为指定文件夹。① 读取指定文件夹下的图像文件信息；② 创建特征张量（值为对应图像的文件名）和标签张量（猫的标签值设置为 0，狗的标签值设置为 1）；③ 返回特征张量和标签张量。

步骤 3 定义解码图像和调整图像大小函数，函数参数为特征张量和标签张量。① 读取原始图像文件；② 解码 JPEG 图像文件；③ 调整图像大小，使其与模型输入要求一致，并进行标准化；④ 返回处理后的图像张量和标签张量。

```
#导入所需要的模块与包
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os

#定义数据读取函数
def read_image_filenames(data_dir):
    cat_dir=data_dir+'cat/'
    dog_dir=data_dir+'dog/'
    #构建特征数据集，值为对应的图片文件名
    cat_filenames=tf.constant([cat_dir+fn for fn in os.listdir(cat_dir)])
```

```

    dog_filenames=tf.constant([dog_dir+fn for fn in os.listdir(dog_dir)])
    filenames=tf.concat([cat_filenames,dog_filenames],axis=-1)
    #构建标签数据集， cat 为 0， dog 为 1
    labels=tf.concat([tf.zeros(cat_filenames.shape,dtype=tf.int32),tf.ones(dog_filenames.shape, dtype=tf.int32)],
axis=-1)
    return filenames,labels
#定义解码图片和调整图片大小的函数
def decode_image_and_resize(filename,label):
    image_string=tf.io.read_file(filename)    #读取原始文件
    #解码 JPEG 图片
    image_decoded=tf.image.decode_jpeg(image_string)
    #调整图像大小，要和模型输入要求一致数，并进行标准化
    image_resized=tf.image.resize(image_decoded,[224,224])/255.0
    return image_resized,label
buffer_size=4000                                #数据缓冲器大小
batch_size=8                                    #批次大小
#定义准备数据集的函数
def prepare_dataset(data_dir):
    filenames, labels = read_image_filenames(data_dir)
    dataset=tf.data.Dataset.from_tensor_slices((filenames,
labels))
    dataset=dataset.map(map_func=decode_image_and_resize,
num_parallel_calls=tf.data.experimental.AUTOTUNE)
    #打乱数据并批量化
    dataset=dataset.shuffle(buffer_size)
    dataset=dataset.batch(batch_size)
    dataset=dataset.prefetch(tf.data.experimental.AUTOTUNE)
    return dataset
#读取训练数据
data_dir='./train/'
dataset_train=prepare_dataset(data_dir)
#将数据集转换为迭代器
it=iter(dataset_train)
images,labels=next(it)
#显示部分样本的图像和标签
plt.rcParams['font.sans-serif']=['SimHei']
for i in range(batch_size):
    plt.subplot(2, 4, i+1)
    plt.title("标签值: "+str(labels.numpy()[i]))
    plt.imshow(images.numpy()[i])
    plt.axis("off")
plt.show()
【运行结果】

```



3. 构建基于 VGG16 的网络模型

猫狗大战网络模型是基于预训练好的 VGG16 网络模型进行构建的。VGG16 网络模型共有 16 层，即 13 个卷积层和 3 个全连接层，不包括池化层和 Softmax 激活函数层。本项目中，冻结卷积基部分，添加分类器部分。新添加的分类器部分包括一个拉伸层、两个全连接层、两个 Dropout 层和一个输出层。由于数据集中有猫和狗两种图像，故应为输出层设置两个神经元，使用 Softmax 函数作为激活函数。

步骤 1 定义构建网络模型的函数。加载预训练好的 VGG16 网络模型，不包括分类器，使用 ImageNet 数据集预训练好的权重，并设置卷积基部分不可训练。

步骤 2 使用函数式 API 模型构建网络模型。除预训练好的 VGG16 网络模型的卷积基部分外，依次添加拉伸层、包含 128 个神经元且使用 ReLU 函数作为激活函数的全连接层、丢弃概率为 0.3 的 Dropout 层、包含 32 个神经元且使用 ReLU 函数作为激活函数的全连接层、丢弃概率为 0.3 的 Dropout 层和包含两个神经元且使用 Softmax 函数作为激活函数的输出层。

步骤 3 创建模型对象，并使用 summary() 函数显示网络模型各层的参数信息。

#构建基于 VGG16 的网络模型

```
def vgg16_model(input_shape=(224,224,3)):
    vgg16=tf.keras.applications.VGG16(include_top=False,weights
    ='imagenet',input_shape=input_shape)          #选择不包含分类器部分
    #设置 VGG-16 预训练模型不可训练
    for layer in vgg16.layers:
        layer.trainable=False
    last=vgg16.output
    #加入剩下未经训练的全连接层
    x=tf.keras.layers.Flatten()(last)
    x=tf.keras.layers.Dense(128,activation='relu')(x)
    x=tf.keras.layers.Dropout(0.3)(x)
    x=tf.keras.layers.Dense(32,activation='relu')(x)
    x=tf.keras.layers.Dropout(0.3)(x)
    x=tf.keras.layers.Dense(2,activation='softmax')(x)
    # 建立新的模型
    model=tf.keras.models.Model(inputs=vgg16.input,outputs=x)
    return model
#创建模型对象并测试
model=vgg16_model()                                #新建模型对象
model.summary()                                     #显示网络模型各层的参数信息
```

4. 编译和训练网络模型

步骤 1 编译网络模型。其中，损失函数使用稀疏交叉熵损失函数，优化器使用 Adam 优化器，性能评估函数使用准确率函数。

步骤 2 训练网络模型。设置批量大小为 8，迭代次数为 5，日志显示模式为 1。

```
#编译网络模型
model.compile(loss='sparse_categorical_crossentropy',optimizer='adam',metrics=['accuracy'])
#训练网络模型
model.fit(dataset_train,batch_size=32,epochs=5,verbose=1)
```

5. 应用网络模型

步骤 1 定义读取测试集图像函数，函数参数为测试集文件夹的路径、读取图像开始位置、结束位置及图像的形状。① 获取测试数据集的文件列表；② 读取测试图像并转换为 NumPy 数组；③ 返回标准化处理后的测试图像数据。

步骤 2 定义预测函数，函数参数为测试集文件夹的路径、读取图像开始位置、结束位置及图像的形状。① 读取测试图像并预处理；② 使用训练好的模型对测试数据进行预测；③ 显示图像及预测结果。

步骤 3 调用预测函数，读取测试数据，使用训练好的模型进行预测，并显示预测结果。

```
#start 代表从第几张开始读取，finish 表示读到第几张为止
def read_image_files(path,start,finish,image_size=(224,224)):
    test_files = os.listdir(path)
    test_images = []
    #读取测试图片并进行预处理
    for fn in test_files[start:finish]:
        img_filename = path+fn
        img=tf.keras.preprocessing.image.load_img(img_filename,target_size=image_size)
        img_array=tf.keras.preprocessing.image.img_to_array(img)
        test_images.append(img_array)
    test_data = np.array(test_images)
    test_data /= 255.0
    return test_data
#定义预测函数
def test_image_predict(path,start,finish,image_size=(224,224)):
    #读取测试图片并预处理
    test_data =read_image_files(path,start,finish,image_size)
    #执行预测
    preds = model.predict(test_data)
    #显示图片及预测结果
    plt.rcParams['font.sans-serif']=['SimHei']
    for i in range(0,finish-start):
        plt.subplot(2, 4, i+1)
        if np.argmax(preds[i])== 0:
            label="预测为猫\n 概率: "+str(preds[i][0])
        else:
            label="预测为狗\n 概率: "+str(preds[i][1])
        plt.title(label)
        plt.imshow(test_data[i])
```

```
plt.axis('off')
plt.tight_layout()
plt.show()
#从 test1 目录下读取训练数据
test_data_dir='./test1/'
test_image_predict(test_data_dir,0,8)
```

【运行结果】



(2) 手写数字图像的生成

1. 准备数据

步骤 1 导入本项目所需要的模块与包。

步骤 2 从 Keras 中导入 MNIST 数据集,将训练集的特征值和标签值分别存储在 x_train 和 y_train 中。

步骤 3 选出训练集中标签值为 8 的图像, 重构数据集, 并随机选取部分图像进行显示。

```
#导入所需要的模块与包
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
import os
import time
from IPython import display
#导入 MNIST 数据集
mnist=tf.keras.datasets.mnist
(x_train,y_train),(_,_)=mnist.load_data()
#重构数据集并显示部分图像
x_train=x_train[y_train==8,]
for i in range(16):
    plt.subplot(4, 4, i+1)
    t=np.random.randint(1,x_train.shape[0])
    plt.axis("off")
    plt.imshow(x_train[t],cmap='gray')
plt.show()
#特征值 x_train 标准化处理
```

```

x_train=x_train.reshape(x_train.shape[0],28,28,1).astype('float32')
x_train=(x_train-127.5)/127.5
#打乱数据并批量化
BUFFER_SIZE=6000
BATCH_SIZE=256
train_dataset=tf.data.Dataset.from_tensor_slices(x_train).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

```

【运行结果】



2. 构建生成模型

生成模型包括一个全连接层、3 个批量标准化层、一个 Reshape 层和 3 个转置卷积层。

步骤 1 定义生成模型函数。

步骤 2 在函数中构建空的顺序网络模型。

步骤 3 为网络模型添加全连接层，结点个数为 12 544 ($7 \times 7 \times 256$)，不使用偏置向量，input_shape 为(100,0)，即输入噪声的大小，使用 LeakyReLU 函数作为激活函数；然后添加批量标准化层。

#构建生成模型

```

def make_generator_model():          #生成模型函数的定义
    model = tf.keras.Sequential()    #构建空的顺序模型
    #为网络模型添加全连接层和批量标准化层
    model.add(tf.keras.layers.Dense(7*7*256,use_bias=False,input_shape=(100,)))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.LeakyReLU())
    #为网络模型添加 Reshape 层
    model.add(tf.keras.layers.Reshape((7,7,256)))
    #为网络模型添加转置卷积层和批量标准化层
    model.add(tf.keras.layers.Conv2DTranspose(128,(5,5),strides=(1,1),padding='same',use_bias=False))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.LeakyReLU())
    #为网络模型添加转置卷积层和批量标准化层
    model.add(tf.keras.layers.Conv2DTranspose(64,(5,5),strides=(2, 2),padding='same',use_bias=False))
    model.add(tf.keras.layers.BatchNormalization())
    model.add(tf.keras.layers.LeakyReLU())
    #为网络模型添加转置卷积层
    model.add(tf.keras.layers.Conv2DTranspose(1,(5,5),strides=(2,2),padding='same',use_bias=False,activation='tanh'))

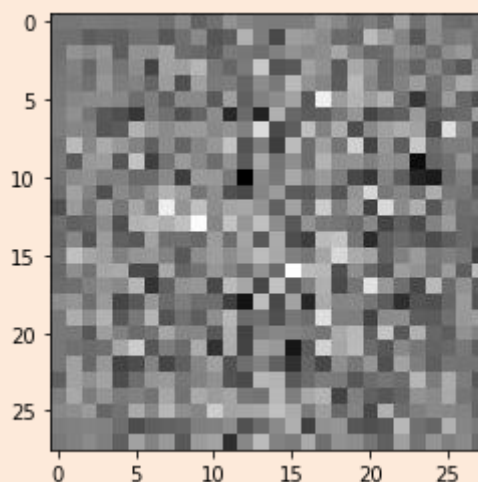
```

```

)
    return model
generator=make_generator_model()      #创建生成模型对象
generator.summary()                   #显示生成模型各层的参数信息
#创建生成样本测试
noise=tf.random.normal([1,100])      #创建输入噪声
#调用生成模型创建生成样本
generated_image=generator(noise,training=False)
#显示生成样本
plt.imshow(generated_image[0,:,:,:0], cmap='gray')

```

【运行结果】



3. 构建判别模型

判别模型包括两个卷积层、两个 Dropout 层、一个拉伸层和一个输出层。

步骤 1 定义判别模型函数。

步骤 2 在函数中构建空的顺序网络模型。

步骤 3 为网络模型添加卷积层，输出通道数为 64，卷积核大小为 5×5 ，步长为(2,2)，填充方式为 'SAME'，input_shape 为(28,28,1)，使用 LeakyReLU 函数作为激活函数；然后添加 Dropout 层，丢弃概率为 0.3。

#构建判别模型

```

def make_discriminator_model():      #判别模型函数的定义
    model=tf.keras.Sequential()      #构建空的顺序模型
    #为网络模型添加卷积层和 Dropout 层
    model.add(tf.keras.layers.Conv2D(64, (5, 5), strides=(2, 2),padding='same',input_shape=[28,28,1]))
    model.add(tf.keras.layers.LeakyReLU())
    model.add(tf.keras.layers.Dropout(0.3))
    #为网络模型添加卷积层和 Dropout 层
    model.add(tf.keras.layers.Conv2D(128, (5, 5), strides=(2,2),padding='same'))
    model.add(tf.keras.layers.LeakyReLU())
    model.add(tf.keras.layers.Dropout(0.3))
    model.add(tf.keras.layers.Flatten())      #为网络模型添加拉伸层
    model.add(tf.keras.layers.Dense(1))      #为网络模型添加输出层
    return model

```



```
discriminator=make_discriminator_model()    #创建判别模型对象
#显示判别模型各层的参数信息
discriminator.summary()
decision=discriminator(generated_image)    #调用判别模型
print(decision)                           #查看判别结果
```

4. 定义损失函数和优化器

步骤 1 定义二元交叉熵损失函数对象 `cross_entropy`。

步骤 2 定义判别模型的损失函数，首先计算真实样本与同维度像素全 1 张量的二元交叉熵损失函数值；然后计算生成样本与同维度像素全 0 张量的二元交叉熵损失函数值；最后将两部分的损失函数值相加作为判别模型损失函数的返回值。

步骤 3 定义生成模型的损失函数，计算生成样本与同维度像素全 1 张量的损失函数值，然后将其作为生成模型损失函数的返回值。

```
#定义损失函数和优化器
cross_entropy=tf.keras.losses.BinaryCrossentropy(from_logits=True)
#定义判别模型的损失函数
def discriminator_loss(real_output,fake_output):
    real_loss=cross_entropy(tf.ones_like(real_output),real_output)
    fake_loss=cross_entropy(tf.zeros_like(fake_output),fake_output)
    total_loss=real_loss+fake_loss
    return total_loss
#定义生成模型的损失函数
def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output),fake_output)
#定义生成模型的优化器
generator_optimizer=tf.keras.optimizers.Adam(1e-4)
#定义判别模型的优化器
discriminator_optimizer=tf.keras.optimizers.Adam(1e-4)
```

5. 定义检查点

对于训练时间较长、计算量较大的项目，在训练过程中，保存阶段性的权重和偏置等参数，可以在训练中途被中断后用来恢复网络模型。

步骤 1 定义检查点文件路径，包括检查点文件存放位置和文件名。

步骤 2 使用 `tf.train.Checkpoint` 类的构造函数创建检查点对象 `checkpoint`，参数为需要保存的对象，此处设置为生成模型的优化器、判别模型的优化器、生成模型对象和判别模型对象。

```
#定义检查点
checkpoint_dir='./checkpoints'    #定义检查点存放位置
#定义检查点的文件名
checkpoint_prefix=os.path.join(checkpoint_dir,"ckpt")
#定义检查点对象
checkpoint=tf.train.Checkpoint(generator_optimizer=generator_optimizer,discriminator_optimizer=discriminator_optimizer,generator=generator,discriminator=discriminator)
```

6. 训练模型

步骤 1 定义训练模型的参数，包括训练次数、输入噪声的长度、生成样本的数量和输入向量。

步骤 2 定义单次训练模型函数 `train_step()`，输入参数为真实样本。① 使用随机函数生成输入噪声；② 使用 `with` 结构记录梯度信息，首先使用生成模型基于输入噪声创建生成样本，然后分别使用判别模型

对真实样本和生成样本进行判别，最后分别计算生成模型和判别模型的损失函数值；③ 计算生成模型和判别模型的梯度；④ 更新生成模型和判别模型的可训练参数。

步骤3 定义生成、显示和保存生成样本图像函数 `generate_and_save_images()`。① 调用函数产生生成样本图像；② 创建子图，在子图中绘制生成样本图像；③ 将生成样本保存到图像文件中。

```
#定义训练模型的参数
EPOCHS=500                                #训练次数
noise_dim=100                             #输入噪音的长度
num_examples_to_generate=16               #生成样本的数量
#产生随机种子作为输入向量
seed = tf.random.normal([num_examples_to_generate, noise_dim])
#定义单次训练模型函数
def train_step(images):
    #使用随机函数生成输入噪声
    noise=tf.random.normal([BATCH_SIZE,noise_dim])
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        #使用生成模型创建生成样本
        generated_images=generator(noise,training=True)
        #使用判别模型对真实样本进行判别
        real_output = discriminator(images, training=True)
        #使用判别模型对生成样本进行判别
        fake_output=discriminator(generated_images,training= True)
        #计算生成模型损失值
        gen_loss=generator_loss(fake_output)
        #计算判别模型损失值
        disc_loss=discriminator_loss(real_output,fake_output)
    #计算生成模型和判别模型的梯度
    gradients_of_generator=gen_tape.gradient(gen_loss,generator.trainable_variables)
    gradients_of_discriminator=disc_tape.gradient(disc_loss,discriminator.trainable_variables)
    #更新生成模型和判别模型的参数
    generator_optimizer.apply_gradients(zip(gradients_of_generator,generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,discriminator.trainable_variables))
#生成、显示和保存生成样本图像
def generate_and_save_images(model,epoch,test_input):
    #产生生成样本
    predictions=model(test_input,training=False)
    #显示生成样本
    fig=plt.figure(figsize=(4,4))
    for i in range(predictions.shape[0]):
        plt.subplot(4,4, i+1)
        plt.imshow(predictions[i, :, :, 0]*127.5+127.5,cmap= 'gray')
        plt.axis('off')
    plt.show()
    #保存生成样本
    plt.savefig('image_at_epoch_{:04d}.png'.format(epoch))
```

#定义完整训练模型函数

```
def train(dataset,epochs):
    for epoch in range(epochs):
        start = time.time()          #开始训练时间
        for image_batch in dataset:  #逐批次训练
            train_step(image_batch)
            #生成、显示和保存生成样本图像
            display.clear_output(wait=True)
            generate_and_save_images(generator,epoch+1,seed)
            #每训练 10 次保存一次检查点
            if(epoch+1)%10==0:
                checkpoint.save(file_prefix = checkpoint_prefix)
            #显示每次训练所花费的时间
            print('第{}次训练花费{}秒'.format(epoch+1, time.time()-start))
train(train_dataset,EPOCHS)        #调用 train()完成模型训练
```

【运行结果】



(3) 交通标志识别

1. 解压和显示数据

步骤 1 导入本项目所需要的模块与包。

步骤 2 解压 traffic.tar 文件到 Jupyter 工作目录下。

步骤 3 定义显示图像函数，参数为存储图像的文件夹。① 显示数据集大小；② 随机抽取图像存入列表；③ 显示列表中的图像。

步骤 4 调用函数显示部分训练图像。

首先，将本次实验的数据“traffic.tar”文件复制到 Jupyter 工作目录中，也可将数据文件放于其他文件夹，如果放于其他文件夹，修改变量 data_set_path（其值为压缩文件的存放路径）的值即可。

【参考代码】

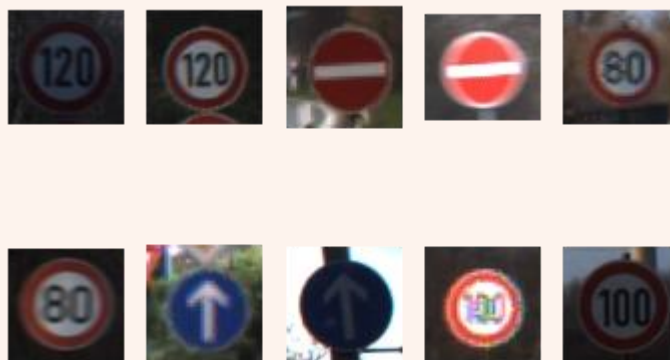
```
#导入所需要的模块与包
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
```

```

import os
import tarfile
#解压文件
data_set_path="./"
archive_file="traffic.tar"
archive_path=os.path.join(data_set_path, archive_file)
tarfile.open(archive_path).extractall(path=data_set_path)
#定义显示图像函数
def display_image(data_dir):
    x_dir=[]
    x_filename=[]
    x_len=0
    #显示数据集大小
    for i in range(5):
        x_dir.append(os.path.join(data_dir,str(i)))
        x_filename.append(os.listdir(x_dir[i]))
        x_len=x_len+len(x_filename[i])
    print("数据集大小为: ",x_len)
    #随机抽取图像存入 image 列表
    image=[]
    for i in range(5):
        for j in range(2):
            t=np.random.randint(0,len(x_filename[i]))
            image.append(x_dir[i]+'/'+x_filename[i][t])
    #显示 image 列表中的图像
    i=1
    for img_path in image:
        plt.subplot(2, 5, i)
        img=mpimg.imread(img_path)
        plt.imshow(img)
        plt.axis('Off')
        i=i+1
    plt.show()
display_image("./traffic/train/") #调用函数显示图像

```

【运行结果】 程序运行结果如图所示。从结果中可以看出，数据集图像质量欠佳，有可能影响训练结果。



2. 数据预处理

步骤 1 对训练集做数据归一化及增强处理，数据增强包括数据旋转、平移、剪切、缩放、翻转等。

步骤 2 对验证集做数据归一化处理。

【参考代码】

#对训练集做数据归一化及增强处理

TRAINING_DIR="./traffic/train/"

training_datagen=tf.keras.preprocessing.image.ImageDataGenerator(

rescale=1./255, #归一化

rotation_range=40, #旋转

width_shift_range=0.2, #水平平移

height_shift_range=0.2, #垂直平移

shear_range=0.2, #剪切

zoom_range=0.2, #缩放

horizontal_flip=True, #水平翻转

fill_mode='nearest')

train_generator=training_datagen.flow_from_directory(TRAINING_DIR,

target_size=(150,150),class_mode='categorical')

#对验证集做数据归一化处理

VALIDATION_DIR="./traffic/validation/"

validation_datagen=tf.keras.preprocessing.image.ImageDataGenerator

(rescale=1./255) #归一化

validation_generator=validation_datagen.flow_from_directory(VALIDATION_DIR,target_size=(150,150),class_mode='categorical')

【运行结果】 程序运行结果如图所示。

```
Found 4200 images belonging to 5 classes.
Found 600 images belonging to 5 classes.
```

ImageDataGenerator 是 tf.keras.preprocessing.image 模块中的图像生成器。它既能够对数据进行增强处理（扩充数据集大小，增加模型的泛化能力），还能自动为训练数据生成标签。flow_from_directory()函数生成归一化及增强后的数据，用于训练或保存。

3. 构建网络模型

交通标志识别网络模型采用卷积神经网络，它包括 4 个卷积层、4 个池化层、6 个 Dropout 层、一个拉

伸层、一个全连接层和一个输出层，网络结构如图所示。由于数据集中有 5 种交通标志图像，故应为输出层设置 5 个神经元，使用 Softmax 函数作为激活函数。

步骤 1 构建卷积神经网络模型对象 model。

步骤 2 在 model 对象中添加第一组“卷积层+池化层+Dropout 层”，首先添加卷积层，输出通道数为 64，卷积核大小为 3×3 ，使用 ReLU 函数作为激活函数，输入数据的形状为(150,150,3)；然后添加最大池化层，池化窗口大小为 2×2 ；最后添加 Dropout 层，丢弃概率为 0.3。

步骤 3 在 model 对象中添加第二组“卷积层+池化层+Dropout 层”，首先添加卷积层，输出通道数为 64，卷积核大小为 3×3 ，使用 ReLU 函数作为激活函数；然后添加最大池化层，池化窗口大小为 2×2 ；最后添加 Dropout 层，丢弃概率为 0.3。

步骤 4 在 model 对象中添加第三组“卷积层+池化层+Dropout 层”，首先添加卷积层，输出通道数为 128，卷积核大小为 3×3 ，使用 ReLU 函数作为激活函数；然后添加最大池化层，池化窗口大小为 2×2 ；最后添加 Dropout 层，丢弃概率为 0.3。

步骤 5 在 model 对象中添加第四组“卷积层+池化层+Dropout 层”，首先添加卷积层，输出通道数为 128，卷积核大小为 3×3 ，使用 ReLU 函数作为激活函数；然后添加最大池化层，池化窗口大小为 2×2 ；最后添加 Dropout 层，丢弃概率为 0.3。

步骤 6 在 model 对象中添加拉伸层。

步骤 7 在 model 对象中添加 Dropout 层，丢弃概率为 0.5。

步骤 8 在 model 对象中添加全连接层，神经元个数为 512，使用 ReLU 函数作为激活函数。

步骤 9 在 model 对象中添加 Dropout 层，丢弃概率为 0.5。

步骤 10 在 model 对象中添加输出层，神经元个数为 5，激活函数采用 Softmax 函数。

步骤 11 使用 summary()函数显示网络模型各层的参数信息。

【参考代码】

```
model=tf.keras.models.Sequential([
    #添加第一组“卷积层+池化层+Dropout 层”
    tf.keras.layers.Conv2D(64,(3,3),activation='relu',input_shape=(150,150,3)),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.3),
    #添加第二组“卷积层+池化层+Dropout 层”
    tf.keras.layers.Conv2D(64,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.3),
    #添加第三组“卷积层+池化层+Dropout 层”
    tf.keras.layers.Conv2D(128,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.3),
    #添加第四组“卷积层+池化层+Dropout 层”
    tf.keras.layers.Conv2D(128,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Dropout(0.3),
    #添加拉伸层和 Dropout 层
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dropout(0.5),
```

```

#添加全连接层和 Dropout 层
tf.keras.layers.Dense(512,activation='relu'),
tf.keras.layers.Dropout(0.5),
#添加输出层
tf.keras.layers.Dense(5,activation='softmax')
])
model.summary() #显示网络模型各层的参数信息

```

【运行结果】 程序运行结果如图所示。

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 64)	1792
max_pooling2d (MaxPooling2D)	(None, 74, 74, 64)	0
dropout (Dropout)	(None, 74, 74, 64)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	36928
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
dropout_1 (Dropout)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
dropout_2 (Dropout)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
dropout_3 (Dropout)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dropout_4 (Dropout)	(None, 6272)	0
dense (Dense)	(None, 512)	3211776
dropout_5 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 5)	2565

Total params: 3,474,501
 Trainable params: 3,474,501
 Non-trainable params: 0

4. 编译、训练和保存网络模型

步骤 1 编译网络模型。其中，损失函数使用交叉熵损失函数，优化器使用 RMSprop 优化器，性能评估函数使用准确率函数。

步骤 2 训练网络模型。其中，迭代次数为 10，日志显示模式为 1。

步骤3 保存网络模型和权重参数到 HDF5 文件中，文件名为“traffic.h5”。

【参考代码】

#编译网络模型

```
model.compile(loss='categorical_crossentropy',optimizer='rmsprop',metrics=['accuracy'])
```

#训练网络模型

```
history= model.fit(train_generator, epochs=10, validation_data =validation_generator,verbose=1)
```

```
model.save("traffic.h5")
```

#保存网络模型

【运行结果】 程序运行结果如图所示。

```
Epoch 1/10
132/132 [=====] - 117s 869ms/step - loss: 1.1145 - accuracy: 0.5017 - val_loss: 0.9406 - val_accuracy: 0.7150
Epoch 2/10
132/132 [=====] - 113s 852ms/step - loss: 0.7981 - accuracy: 0.6338 - val_loss: 0.5861 - val_accuracy: 0.7367
Epoch 3/10
132/132 [=====] - 113s 858ms/step - loss: 0.7236 - accuracy: 0.6650 - val_loss: 0.6386 - val_accuracy: 0.7167
Epoch 4/10
132/132 [=====] - 117s 885ms/step - loss: 0.6760 - accuracy: 0.6814 - val_loss: 0.5406 - val_accuracy: 0.8317
Epoch 5/10
132/132 [=====] - 116s 878ms/step - loss: 0.6271 - accuracy: 0.7007 - val_loss: 0.6120 - val_accuracy: 0.8133
Epoch 6/10
132/132 [=====] - 115s 872ms/step - loss: 0.6120 - accuracy: 0.7179 - val_loss: 0.5621 - val_accuracy: 0.7783
Epoch 7/10
132/132 [=====] - 112s 846ms/step - loss: 0.6010 - accuracy: 0.7176 - val_loss: 0.5893 - val_accuracy: 0.6933
Epoch 8/10
132/132 [=====] - 113s 854ms/step - loss: 0.5867 - accuracy: 0.7329 - val_loss: 0.5341 - val_accuracy: 0.8017
Epoch 9/10
132/132 [=====] - 114s 861ms/step - loss: 0.5925 - accuracy: 0.7395 - val_loss: 0.6074 - val_accuracy: 0.7700
Epoch 10/10
132/132 [=====] - 112s 848ms/step - loss: 0.5630 - accuracy: 0.7460 - val_loss: 0.5886 - val_accuracy: 0.7833
```

tf.keras.Model 类将定义好的网络模型封装入一个对象，用于训练、测试和预测。在该类中，训练好的网络模型、权重等可以保存到 HDF5 文件中，常用函数如下。

(1) save(filepath)函数用于将 Keras 模型和权重保存到 filepath 所指定的 HDF5 文件中，该文件将包括模型的结构、权重、训练配置（损失函数和优化器等）及优化器的状态等。tf.keras.models.load_model(filepath)函数用于使用已保存的 HDF5 文件实例化网络模型。

(2) save_weights(filepath)函数用于保存模型的权重到 filepath 所指定的 HDF5 文件中。

(3) load_weights(filepath)函数用于使用 filepath 保存的权重初始化模型。

5. 可视化训练的结果

步骤1 读取 history 的 history 属性，分别将训练集损失函数值赋值给变量 loss、训练集准确率赋值给变量 acc、验证集损失函数值赋值给变量 val_loss、验证集准确率赋值给变量 val_acc。

步骤2 创建子图，在子图 1 中绘制损失函数值的折线图。

步骤3 在子图 2 中绘制准确率的折线图。

【参考代码】

#读取 history 的 history 属性

```
loss=history.history['loss']
```

#训练集损失函数值

```
acc=history.history['accuracy']
```

#训练集准确率

```
val_loss=history.history['val_loss']
```

#验证集损失函数值

```
val_acc=history.history['val_accuracy']
```

#验证集准确率

#创建画布并设置画布的大小

```
plt.figure(figsize=(10,3))
```

#在子图 1 中绘制损失函数值的折线图

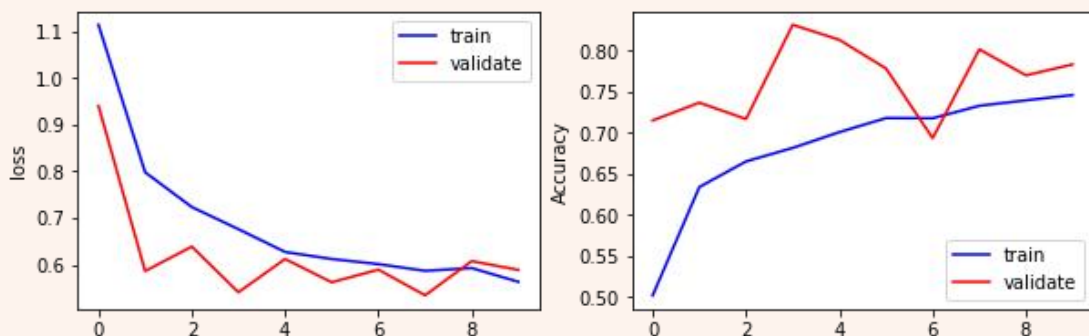
```
plt.subplot(121)
```

```
plt.plot(loss,color='b',label='train')
```



```
plt.plot(val_loss,color='r',label='validate')
plt.ylabel('loss')
plt.legend()
#在子图 2 中绘制准确率的折线图
plt.subplot(122)
plt.plot(acc,color='b',label='train')
plt.plot(val_acc,color='r',label='validate')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

【运行结果】 程序运行结果如图所示。



6. 应用网络模型

步骤 1 定义读取测试集图像函数，函数参数为测试集文件夹的路径、读取图像开始位置、结束位置及图像的形状。① 获取测试集的文件列表；② 读取测试图像并转换为 NumPy 数组；③ 返回标准化处理后的测试图像数据。

步骤 2 定义预测函数，函数参数为测试集文件夹的路径、读取图像开始位置、结束位置及图像的形状。

① 读取测试图像并进行预处理；② 使用训练好的模型对测试数据进行预测；③ 显示图像及预测结果。

步骤 3 调用预测函数，读取测试数据进行预测并显示预测结果。

【参考代码】

```
#定义读取测试集图像函数
def read_image_files(path,start,finish,image_size=(150,150)):
    test_files=os.listdir(path)
    test_images=[]
    #读取测试图像
    for fn in test_files[start:finish]:
        img_filename=path+fn
        img=tf.keras.preprocessing.image.load_img(img_filename, target_size=image_size)
        img_array=tf.keras.preprocessing.image.img_to_array(img)
        test_images.append(img_array)
    test_data=np.array(test_images)      #测试图像预处理
    test_data/=255.0
    return test_data
#定义预测函数
```

```

def test_image_predict(path,start,finish,image_size=(150,150)):
    #读取测试图像并预处理
    test_data=read_image_files(path,start,finish,image_size)
    #执行预测
    preds=model.predict(test_data)
    #显示图像及预测结果
    plt.rcParams['font.sans-serif']=['SimHei']
    for i in range(0,finish-start):
        plt.subplot(2,4,i+1)
        if np.argmax(preds[i])==0:
            label="预测值： 限速 120\n 概率： "+str(preds[i][0])
        elif np.argmax(preds[i])==1:
            label="预测值： 禁行\n 概率： "+str(preds[i][1])
        elif np.argmax(preds[i])==2:
            label="预测值： 限速 80\n 概率： "+str(preds[i][2])
        elif np.argmax(preds[i])==3:
            label="预测值： 直行\n 概率： "+str(preds[i][3])
        else:
            label="预测值： 限速 100\n 概率： "+str(preds[i][4])
        plt.title(label)
        plt.imshow(test_data[i])
        plt.axis('off')
    plt.tight_layout()
    plt.show()
#读取测试数据并预测
test_data_dir='./traffic/test/'
test_image_predict(test_data_dir,0,8)

```

【运行结果】 程序运行结果如图所示。从结果中可以看出，该模型所识别的交通标志大部分是正确的。

